Week 2 Lab Exercise

Please follow these steps for the lab exercise. This is particularly important for students who have never used Laravel previously. It is **highly recommended** that you do some vanilla Laravel work before undertaking the main assignment. This will help to familiarise you with folder structure and a basic understanding of how Laravel works.

This exercise will use Codespaces and will require a GitHub account. You should already have one of these.

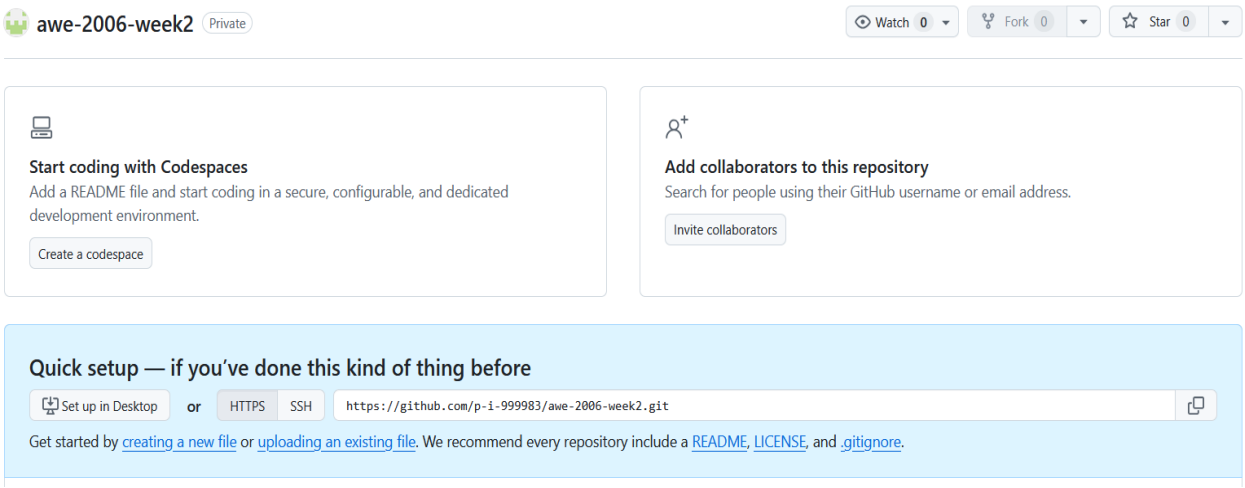
It is useful to have the Laravel documentation site, open and viewable. Its location is https://laravel.com/framework/docs/13.x/

Some files are provided in the accompanying zip file. Other changes you will need to edit files in the project space.

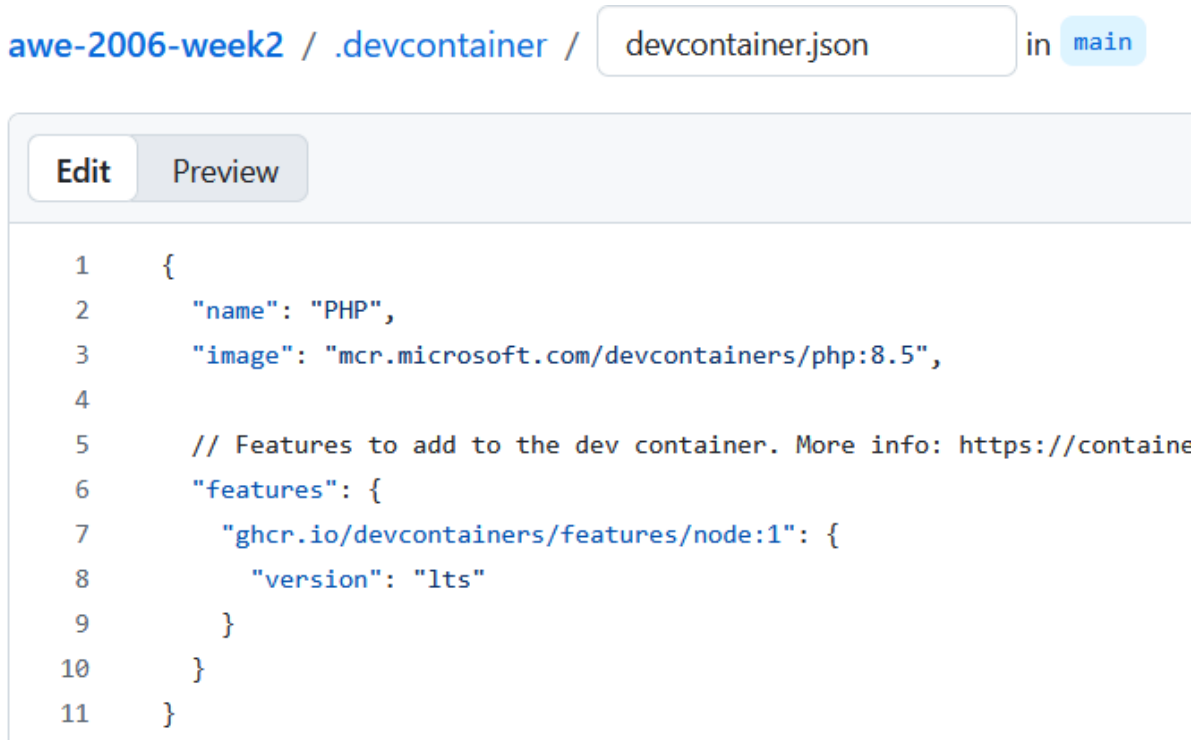
Steps:

1. Create a new GitHub repository. This will be one that you own (i.e. in your own space). It can be private. Initialise it as an empty repository.

2. Having created a new repository – GitHub will want you to add some content. As we will be using Codespaces and will require our container to have particular software – choose the option '**creating a new file**'



3. This is .devcontainer/devcontainer.json (a copy is in the zip file)



This container file will install Ubuntu with php8.5 on your Codespace container. Because this contains PHP it will also include Composer. The 'features' requests that Node is also installed.

You can check this with the following commands:

php -v
composer -V
node -v

```
@p-i-999983 ➜/workspaces/awe-2006-week2 (main) $ php -v
PHP 8.5.10 (cli) (built: Aug 31 2026 19:13:23) (NTS)
Copyright (c) The PHP Group
Built by https://github.com/docker-library/php
Zend Engine v4.5.10, Copyright (c) Zend Technologies
    with Xdebug v3.5.3, Copyright (c) 2002-2026, by Derick Rethans
    with Zend OPcache v8.5.10, Copyright (c), by Zend Technologies
@p-i-999983 ➜/workspaces/awe-2006-week2 (main) $ composer -V
Xdebug: [Step Debug] Could not connect to debugging client. Tried: localhost:9003 (through xdebug.client_host/xdebug.client_port).
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.5.10 (/usr/local/bin/php)
Run the "diagnose" command to get more detailed diagnostics output.
@p-i-999983 ➜/workspaces/awe-2006-week2 (main) $ node -v
v24.21.0
```

4. To install Laravel into this folder. Please note the following steps. We want Laravel in this folder – but Laravel installation requires it to be placed in

a non-empty folder, and our folder does contain a .devcontainer folder. What we will do, is to install in a temp folder, move the contents into this folder, and then delete the temp folder.

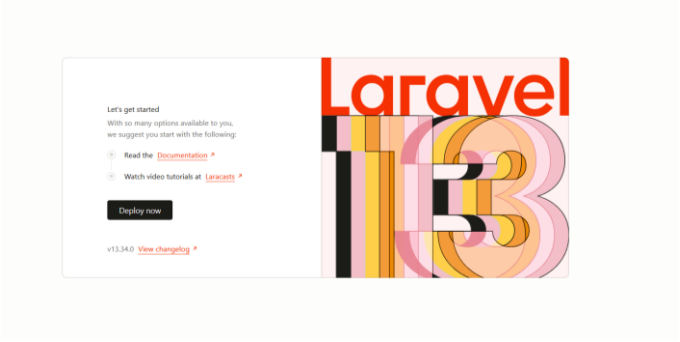
```
composer create-project laravel/laravel temp-laravel
mv temp-laravel/* temp-laravel/.*  .
rmdir temp-laravel
```

It is worthwhile looking through the installation chain of events when installing as you will see:

```
copy('.env.example', '.env');
Updating dependencies
Installing dependencies from lock file
artisan key:generate
artisan migrate
```

These are all steps that form part of our own start sequence when working with the main assignment for the first time.

5. Try **`php artisan serve`** : (Ctrl-C to exit): Our site is viewable:



As we are on Codespaces, it is better to use

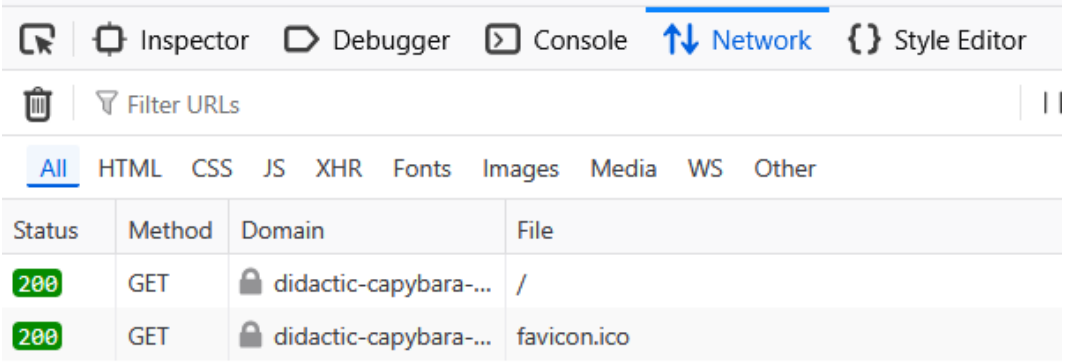
`php artisan serve --host=0.0.0.0 --port=8000`

This will help to resolve paths and links when pages, css and js are linked to at a later point. Remember Codespaces is running on a container in the cloud. Forwarding rules are slightly different.

6. First Question:

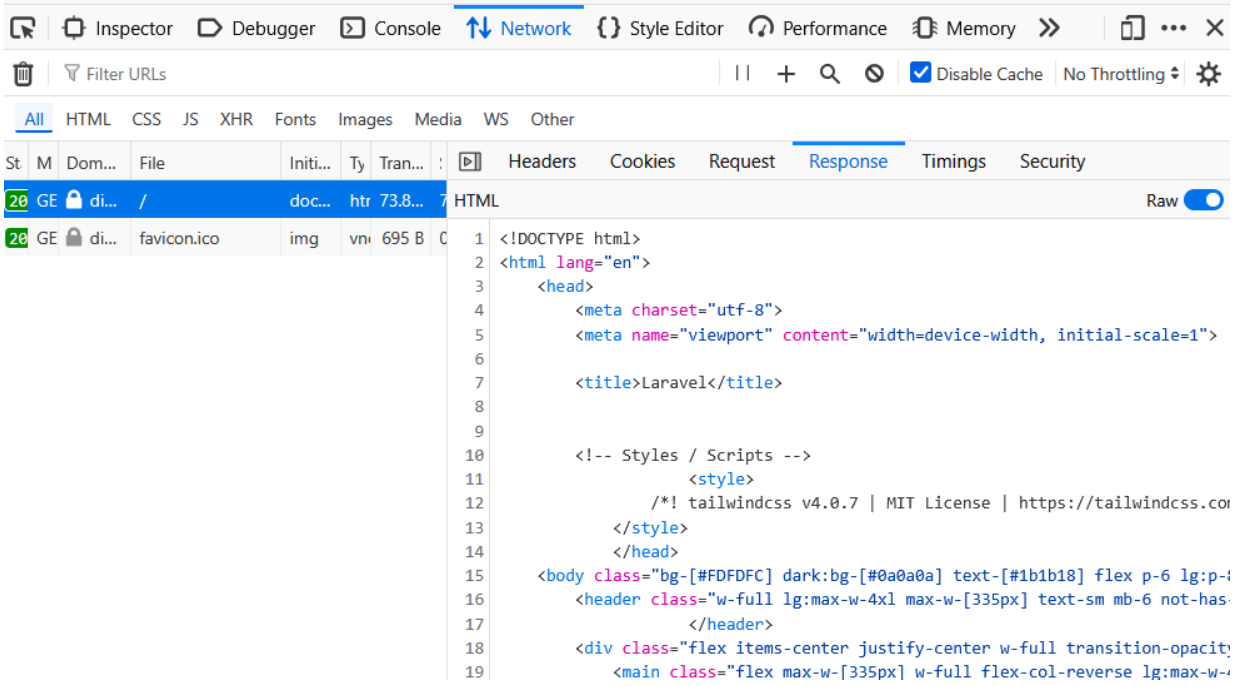
Why does this page appear? Let us use our knowledge of http and trace a path through.

BDT (Browser Developer Tools)

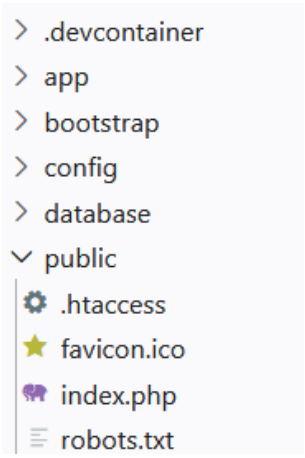


The network tab says that the / location returned a 200. Remember a webserver will typically look for index.(html,php) at this location if a name is not specified.

The response shows that HTML is returned:



Back to Codespaces and our file structure – we have an index.php



It's a very simple file in appearance:

public > index.php > ...

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Http\Request;

define('LARAVEL_START', microtime(true));

// Determine if the application is in maintenance mode...
if (file_exists($maintenance = __DIR__.'/../storage/framework/maintenance.php')) {
    require $maintenance;
}

// Register the Composer autoloader...
require __DIR__.'/../vendor/autoload.php';

// Bootstrap Laravel and handle the request...
/** @var Application $app */
$app = require_once __DIR__.'/../bootstrap/app.php';

$app->handleRequest(Request::capture());
```

But look at the last line.....

Our app (Laravel), has a method (handleRequest) which takes a parameter (the HTTP request). The result of this method will be the output of index.php

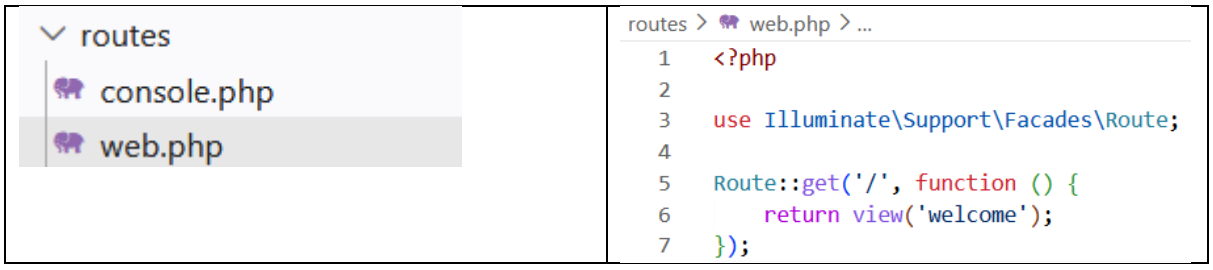
Now – have a look at .htaccess

public > .htaccess

```
1   <IfModule mod_rewrite.c>
2       <IfModule mod_negotiation.c>
3           Options -MultiViews -Indexes
4       </IfModule>
5
6       RewriteEngine On
7
8       # Handle Authorization Header
9       RewriteCond %{HTTP:Authorization} .
10      RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
11
12      # Handle X-XSRF-Token Header
13      RewriteCond %{HTTP:x-xsrf-token} .
14      RewriteRule .* - [E=HTTP_X_XSRF_TOKEN:%{HTTP:X-XSRF-Token}]
15
16      # Redirect Trailing Slashes If Not A Folder...
17      RewriteCond %{REQUEST_FILENAME} !-d
18      RewriteCond %{REQUEST_URI} (.+)/$
19      RewriteRule ^ %1 [L,R=301]
20
21      # Send Requests To Front Controller...
22      RewriteCond %{REQUEST_FILENAME} !-d
23      RewriteCond %{REQUEST_FILENAME} !-f
24      RewriteRule ^ index.php [L]
25  </IfModule>
```

.htaccess allows for rules to be defined on how the traditional filename approach of a URL is handled. The REWRITE rule here states that whatever the /######## is on the URL, rewrite it ti point to index.php (lines 22-24). Rewriting this doesn't remove this information – it is still part of the HTTP message. It just means that all URLS to this address are **routed** to index.php. This means that the application will decide how toproceed with this information. The URL becomes an **instruction** rather than a **file location**.

This instruction is defined in web.php (routes directory)



On line 5 of web.php, change / to /one. Reload the page in the browser. It will return a 404, but if /one is appened to the URL – it will return the welcome page again.

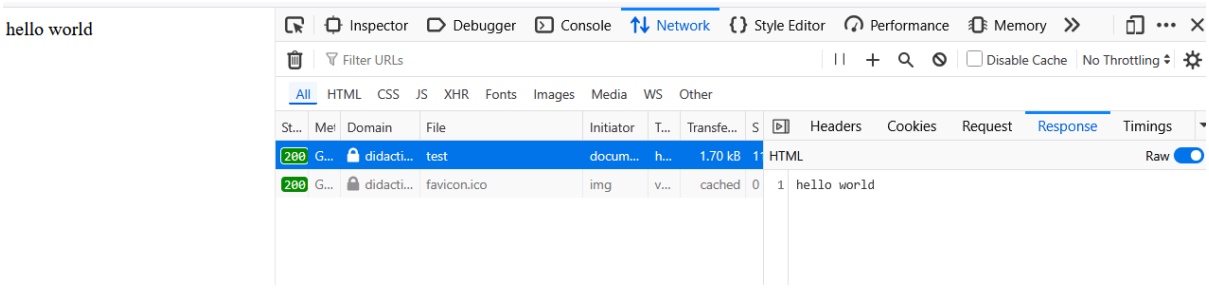
All URLS get routed to this location (note that this isn't strictly true as Laravel will *reserve* some for special use cases, but at this time consider it to be the case!)

So / will return a view called welcome.

In the resources directory, you will find a views folder with a file called '**welcome**.blade.php'. This is what is being returned here. It is an example of a html file being returned directly. Note that when more complex tasks are required, several further steps will take place between capturing the route and returning the **response**

7. To build your own route and response (at the most simple level). Add the following to web.php

```
9  Route::get('/test', function () {
10     return "hello world";
11 });
```



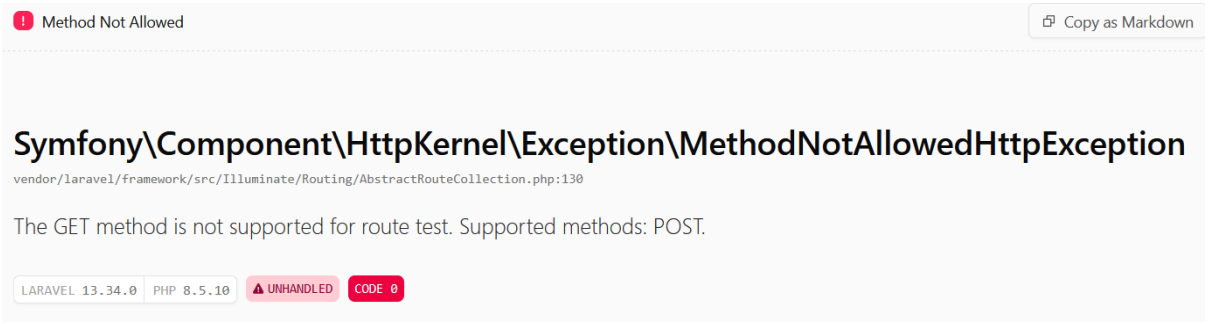
Note that the response – as seen in the BDT – shows that it is unmarked up text.

8. Second Question:

What is the significance of Route:**get** in the line?

This indicates the ACTION that the HTTP request is associated with. Change the line, such that it is:

```
Route::post('/test', function () {
```



When the route was not defined at all a 404 was returned. Now the error message indicates that there is a route at this point, but not associated with this ACTION.

We can quickly see the route working with a CURL command in the terminal window. (127.0.0.1 - should still work here, although you could replace with your actual codespaces URL)

```
curl --request POST http://127.0.0.1:8000/test
```

While I say – we will see it work – Laravel protects all POST behind csrf tokens. At this point your response in the terminal indicate a page expired. You will need to make one quick change to make Hello World appear.

In bootstrap/app.php;

```
14        ->withMiddleware(function (Middleware $middleware): void {
15            $middleware->preventRequestForgery(except: [
16                '/test'
17            ]);
18        })
```

```
@p-i-999983 ➜/workspaces/awe-2006-week2 (main) $ curl --request POST http://127.0.0.1:8000/test
hello world
```

9. While the following steps are not how you would do this in the real world. The purpose is to think HTTP protocol more. Make the following changes to web.php

```
use Illuminate\Http\Request;     //at the top, needed as the Request class will be used.

Route::post('/', function (Request $request) {

    if($request->isJson()) {
        return $request->input('name');
    }
    return $request->role;


});
```

```
Route::get('/person/{id}', function(Request $request, int $id){
        return view('person', ['data'=>$request->all(), 'id'=>$id]);
});
```

In the resources/views folder, create person.blade.php file: (in zip file)

```
<!DOCTYPE html>
<html lang='en'>
    <head>
        <meta charset="utf-8">
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">

        <script src="https://cdn.tailwindcss.com"></script>

        <title>My test person page</title>

    </head>
    <body class="m-5">
    <h1 class="text-xxl">Person ID {{$id}}</h1>

    <div class="w-full max-w-sm bg-slate-50 border border-2 border-gray-400 rounded-lg shadow dark:bg-gray-800 dark:border-gray-700">
    <div class="flex flex-col items-center pt-10 pb-10">
        <img class="w-24 h-24 mb-3 rounded-full shadow-lg" src="/images/pi.webp" alt="Person Image"/>
        <h5 class="mb-1 text-xl font-medium text-gray-900 dark:text-white">{{$data['name']??'UNKNOWN'}}</h5>
        <span class="text-base text-gray-500 dark:text-gray-400">{{$data['role']??'UNKNOWN'}}</span>
    </div>
</div>

</body>
</html>
```

Look at responses for:

CURL commands: (you will need to add the identified route to the app.php file or you will get a token issue)

```
curl --header "Content-Type: application/json"   --request POST
   --data '{"name":"Patrick","role":"Lecturer","email":"p@p"}'
   http://127.0.0.1:8000
```

```
curl --header "Content-Type: application/x-www-form-urlencoded"
   --request POST   --data
   'name=Patrick%20Ingham&role=Lecturer&email=p@p'
   http://127.0.0.1:8000
```

And then in the browser:
/person/2?name=Patrick&role=Lecturer

